Date: 8/10/25

Task 12: Simulate Gaming Concepts using Pygame.

Problem 11: write a python program to create to a snake game using Python.

Aim: To Simulate gaming concepts using python.

Algorithm:

1) Import Pygame package and initialize it
2) Define the window size and title.
3) Create a Snake class which initalizes the snake position, color, and move ment.
4) Create a fruit class which initialize the fruit position and color.
5) Creak a function to check it the snake collides with the fruit and increase the score.
6) creat a function to check if the snake collides with window and end the game.
7) Create a function to update the game display and draw the snake and fruit.
8) Create a function to update the game display and draw the snake and fruit.
9) create a game loop to continuosly update to the game display snake position and check for collisions.
10) End the game of the war 2 units are the snake collides with the window.

Program:

```
import pygame
    import time
    import random

    Snake_Speed = 15
    window_x = 720
    window_y = 480
```

Output:

Score: 0.

```
black = pygame.color(0,0,0)
white = pygame.color(255,255,255)
red = pygame.color(255,0)
green = pygame.color(0,255,0)
blue = pygame.color(0,0,255)
pygame.init()
pygame.display.set_captan("Greeks for greek shakes")
game_window = pygame.display.set mode
                    ((window_x, window_y))
fps = py game.time.Clock()
Snake_position = [100,50]
snake_body = [[100,50],
              [90,50],
              [80,50],
              [70,50],
             ]
fruit_position = [random.rand range(1,(window_x//10))*
                  10, random.rand range(1,(window_y//10))*]
fruit_spawn = True
direction = "RIGHT"
change_to = direction
score = 0
def show_score (choice, color, font, size):
    Score_font = pygame font.sys.font (font, size)
    Score_surface = score_font.render ['score", "str(score),
                                       True, color]
    Score_rect = seore_surface get_rect()
    game_windaw.blit (score_surface, score_rect)
def game_over ():
```

```
my_font = pygame.font.SysFont('times new roman', 50)
game_over_surface = my_font.render(
    'Your Score is: ' + str(score), True, red)
game_over_rect = game_over_surface.get_rect()
game_over_rect = midtop = (window_x/2, window_y/4)
game_window.blit(game_over_surface, game_over_rect)
pygame.display.flip()
time.sleep(2)
pygame.quit()
quit()
while true:
for event in pygame.event.get():
if event.type == pygame.KEYDOWN:
    if event.key == pygame.K_UP:
        change_to = 'UP'
if event.key == pygame.K_DOWN:
        change_to = 'DOWN'.
if event.key == pygame.K_LEFT:
        change_to = 'LEFT'
if event.key == pygame.K_RIGHT:
        change_to = 'RIGHT'
if direction == 'UP':
        snake_position[1] = 10
    if direction == 'DOWN':
        snake_position[1] += 10
    if direction == 'LEFT':
        snake_position[0] -= 10
    if direction == 'RIGHT':
        snake_position[0] += 10
snake_body.insert(0, list(snake_position))
if snake_position[0] == fruit_position[0] and
        snake_position[1] == fruit_position[1]:
    score += 10.
```

```
    fruit = spawn = false
else:
    snake_body.pop()
    if not fruit_spawn:
fruit_position = [random.randrange(1,(window_x//10))*10,
    random.randrange(1,(window_y//10))*10]
        fruit_spawn = True
    game_window.fill(black)
    for pos in snake_body:
    Pygame.draw.rect(game_window, green, pygame.Rect
                    (pos[0], pos[1],10,10))
pygame.draw.rect(game_window, white, pygame.Rect(fruit
_position[0], fruit_position[1],10,10))
if snake_position[0]<0 or snake_position[0]>window_x-10
    game_over()
if snake_position[1]<0 or snake_position[1]>window_y-10
    game_over()
for block in snake_body[1:]:
        if snake_position[0]==block[0] and snake_position
                                        [1]==block[1]
            game_over()
show_score(1,white, 'times new roman',20)
pygame.display.update()
fps.tick(snake_speed).
```

Problem: 12-2:

Aim:- write a python program to develop achess board using pygame.

Algorithm:

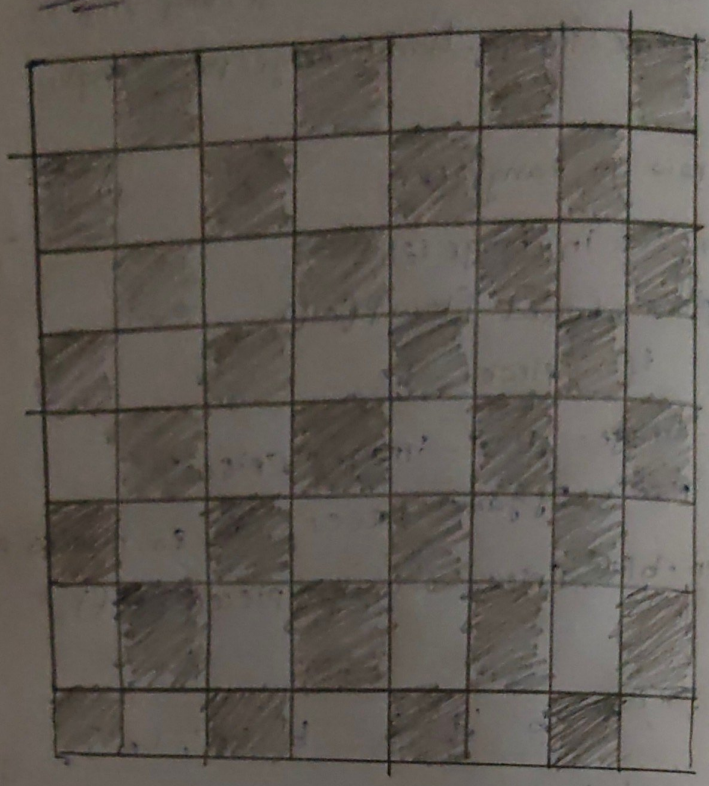
1) import pygame and initialize it.
2) set screen size and title.
3) Define colors for the board and pieces.
4) Define a function to draw the board by coping over work and columns and drawing squares of different colors.
5) Define a function to draw the pieces on the board by loading images for each pieces and placing down them on the corresponding squares.
6) Define the initial state of the board as a list of lists containing the pieces.
7) Draw the board and pieces on the screen.
8) start the game loop.

Program:

```
import pygame

  pygame.init()

screen-size = (640, 640)

screen = pygame.display.set-mode (screen-size)

 pygame = display.set-caption('chess board')

  black = (0,0,0)

 white = (255,255,255)
 brown = (153, 76, 0)
 def draw-board():

      for row in range(8):

        for col in range(8):
   square-color = white if (row+col)%2==0 else brown
square-rect = pygame.Rect(col*80, row*80, 80, 80)
pygame.draw.rect(screen, square-color, square-rect)
```

```
def draw - pieces (board):
    piece - images = {
'r' = pygame . image . load ('image / rook . png'),
'n' = pygame . image . load ('image /k night . png'),
'l' : pygame . image load (' image /bi shop. png'),
'q' : pygame . image . load ('image/queen . png'),
'k' : pygame . image . load ('image / king . png'),
'p' : pygame . image . load ('image/ pawn . Png'),
}
for row in range (8):
  for col in range (8):
    piece = board [row] [col]
      if piece != ' ':
piece - image = piece - images [pieces]
piece - rect = pygame . RECT [col * 80, row * 80, 80, 80]
 screen . blit (piece - image, piece - rect)
board = [
    ['r', 'n', 'b', 'q', 'k', 'b', 'n', 'r'],
  ['p', 'p', 'p', 'p', 'p', 'p', 'p', 'p'],
   [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
  [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
   ['P', 'P', 'P', 'P', 'P', 'P', 'P', 'P'],
]
  draw - board ()
  draw - pieces [board]
while true;
    for event in pygame . event . get ():
      if event type == pygame . quit.
        pygame . quit()
        quit ()
    pygame . display . update ().
```

output :-



completed

| VEL TECH | |
|---|---|
| EX NO. | 12 |
| PERFORMANCE (5) | 5 |
| RESULT AND ANALYSIS (5) | 5 |
| VIVA VOCE (5) | 5 |
| RECORD (5) | |
| TOTAL (20) | 15 |

Result: Thus the program for pygame is executed and verified successfully.